Product Requirement Document (PRD)

CatalogGuard AI - MVP

Sprint	1	**Theme**	Marketplace Catalog Validation
Squad	54	**Date**	05/08/2026
Team Members	S P Jyotiranjan Sahoo, Mohammed Yaseen		

1. Project Overview

CatalogGuard AI is a web application that checks seller product feeds before they reach customers.

Sellers upload product data in CSV format. The system validates required fields, prices, inventory, categories, and duplicate SKUs. AI checks whether a product's title and category match and suggests clear corrections.

The application will use Next.js and TypeScript for the desktop application and backend-for-frontend, MongoDB for operational data, and FastAPI/Python for the validation pipeline. LangGraph and LangChainJS will run the controlled AI orchestration layer. Every service will emit correlated structured logs to one centralized log stream.

2. Problem Statement

A multi-vendor marketplace receives product listings, pricing updates, and inventory feeds from thousands of sellers daily, but no validation workflow detects inconsistent catalog data before customer-facing errors appear.

Common issues include missing product details, incorrect prices, negative stock, duplicate SKUs, and products placed in the wrong category.

The proposed workflow is:

> **Seller CSV** > **Next.js Ingest** > **FastAPI Validation** > **AI Orchestration** > **Review** > **Approval**

3. Objectives

The MVP will:

- Accept seller product feeds in CSV format.
- Detect missing or invalid catalog data.
- Flag incorrect prices, stock values, and duplicate SKUs.
- Use AI to check title and category consistency.
- Give sellers simple correction suggestions.
- Provide a dashboard for reviewing and approving records.
- Store products, issues, and review decisions in MongoDB.

4. Target Users

- **Sellers:** Upload feeds and view validation errors.
- **Catalog Reviewers:** Review, approve, or reject flagged products.
- **Administrators:** View overall catalog quality and manage users.

5. Proposed Solution

5.1 Feed Upload

Sellers will upload a CSV file containing:

- SKU

- Product title
- Description
- Category
- Price
- Inventory quantity

The Next.js backend will authenticate the seller, validate the file type and size, create a feed ID and correlation ID, and store the original file plus processing status. It then submits a service-authenticated validation job to FastAPI.

5.2 Rule-Based Validation (FastAPI)

The system will check:

- Missing required fields
- Invalid price values
- Negative inventory
- Duplicate SKUs
- Invalid data types
- Unsupported categories
- Empty titles or descriptions

Each record will be marked as **Valid**, **Warning**, or **Error**.

FastAPI is the authoritative validation service. It normalizes rows, applies deterministic rules, produces structured issue records, and returns a signed internal callback to the Next.js backend. The Next.js backend verifies the callback and updates MongoDB atomically with the feed status, product records, and validation results.

5.3 AI Orchestration (LangGraph and LangChainJS)

AI will only be used where deterministic validation rules are insufficient. FastAPI owns the validation job and invokes an internal Node AI-orchestration service because LangChainJS is a JavaScript library. This keeps the AI step behind the FastAPI pipeline while using the correct runtime for LangGraph and LangChainJS.

The workflow checks whether the product title, description, and category match; suggests a better category when a mismatch is found; and converts technical validation errors into clear correction messages.

LangGraph controls the AI workflow state: eligible record selection, structured-output request, schema validation, retry or graceful fallback, and return of an advisory result. LangChainJS performs the model call and parses a constrained response containing suggestion, confidence, and evidence. FastAPI validates that response before it is sent back to the Next.js backend.

AI suggestions will not automatically approve products. A reviewer will make the final decision.

5.4 Centralized Logs and Service Connection

Next.js, FastAPI, and the Node AI-orchestration service will use service-to-service authentication and a shared correlation ID. Each request and job transition will log feed ID, product ID where applicable, actor type, route or workflow node, duration, outcome, and safe error code. OpenTelemetry will propagate trace context and structured logs will be collected centrally in Grafana Loki, making one feed traceable across upload, validation, AI suggestion, reviewer decision, and callback processing.

5.5 Review Workflow

Reviewers can:

- View flagged products.
- Compare submitted data with suggested corrections.
- Approve or reject a product.
- Add a short review note.
- Re-run validation after a correction.

5.6 MVP Scope

The MVP will not include live seller APIs, automated marketplace publishing, image analysis, email notifications, advanced duplicate detection, or complex analytics.

These features can be added after the MVP.

6. Key Features

Feature	Description
CSV Upload	Upload seller product feeds
Data Validation	Check required fields, prices, stock, and SKUs
AI Category Check	Detect category and product-content mismatches
Correction Suggestions	Provide understandable fixes
Review Queue	Display products requiring attention
Approval Workflow	Approve or reject flagged records
Dashboard	Show upload and validation statistics
Audit History	Store review decisions and notes

7. Technology Stack

Technology	Purpose
Next.js + TypeScript	Desktop UI, authentication, role-aware backend-for-frontend, upload API, and internal result callback
FastAPI + Python	Authoritative CSV normalization, deterministic validation, validation-job lifecycle, AI-result validation, and service callbacks
LangGraph	Stateful AI workflow orchestration: eligibility, tool flow, retries, fallback, and human-review handoff
LangChainJS	Node AI-orchestration runtime for structured LLM calls, prompt/tool handling, and constrained output parsing
MongoDB	Store users, products, uploads, and validation results
Mongoose	MongoDB schema management
OpenAI API	Category checks and correction suggestions through the LangChainJS workflow
OpenTelemetry + Grafana Loki	Correlation IDs, trace-context propagation, and centralized structured logs across Next.js, FastAPI, and AI services
Zod	Server and form validation
Pydantic	FastAPI request, response, and AI-result schema validation
Tailwind CSS	User-interface styling
Recharts	Basic dashboard charts
Auth.js	User login and role management
GitHub	Version control and collaboration

8. Functional Requirements

The system should be able to:

1. Allow users to log in.
2. Allow sellers to upload CSV files.
3. Read and store product records.
4. Validate required product fields.
5. Detect invalid prices and inventory.
6. Detect duplicate SKUs within an upload.
7. Use AI to check category consistency.
8. Display errors and suggested corrections.
9. Allow reviewers to approve or reject products.
10. Display basic upload and validation statistics.
11. Store all review decisions in MongoDB.

9. Non-Functional Requirements

The application should be:

- Easy to understand and navigate.
- Secure and accessible only to logged-in users.
- Able to process an MVP batch of up to 1,000 products.
- Reliable when the AI service is unavailable.
- Responsive on desktop and tablet screens.
- Structured so more validation rules can be added later.

10. Proposed Dashboard Flow

Dashboard 1 - Overview

- Total uploads
- Total products processed
- Valid products
- Flagged products
- Approved and rejected products

Dashboard 2 - Upload and Results

- CSV upload form
- Upload status
- Validation summary
- Error table
- Downloadable error report

Dashboard 3 - Review Queue

- Flagged product list
- Original product information
- Validation errors
- AI category suggestion
- Approve and reject actions

11. User Flow

Login > **Upload CSV Feed** > **Run Validation** > **Run AI Category Check** > **Display Valid and Flagged Products** > **Reviewer Approves or Rejects** > **Store Final Decision**

12. Project Delivery

The complete project will be delivered as one finished submission. The work includes:

- Finalizing the database structure.
- Building authentication and user roles.
- Creating the CSV upload screen.
- Storing product records in MongoDB.
- Implementing rule-based validation.
- Displaying validation results.
- Adding AI category checks and correction suggestions.
- Building the review queue.
- Adding approve and reject actions.
- Creating the overview dashboard.
- Testing the complete workflow.
- Deploying the MVP.
- Completing the project documentation.

The final deliverables are:

- This Product Requirement Document (PRD), delivered in PDF format
- Basic UX mock-up
- Working Next.js application
- MongoDB database
- GitHub repository
- Setup and deployment instructions

13. Success Criteria

The project will be successful when:

- A seller can upload a valid CSV feed.
- The system detects missing fields, invalid prices, negative stock, and duplicate SKUs.
- AI identifies obvious category mismatches and suggests corrections.
- Reviewers can approve or reject flagged products.
- Validation and review results are stored in MongoDB.
- The dashboard displays correct record counts.
- A batch of 1,000 products can be processed without data loss.
- The application is deployed and demonstrated within two weeks.

14. Expected Outcome

The final MVP will validate seller product feeds before publication, give reviewers one place to manage catalog errors, and prevent incorrect catalog data from reaching customers.

The completed workflow will be:

CSV Upload > **Rule Validation** > **AI Category Check** > **Human Review** > **Approval**